

TETHERLESS: SECURITY & ENGINEERING AUDIT REPORT

Comprehensive Independent Architecture, Cryptographic & Source Code Implementation Review

TARGET SYSTEM

Tetherless (v1.0-SNAPSHOT)

MODULES IN AUDIT

core-shared, chat-server,
chat-desktop

AUDIT DATE

September 2026

DEPLOYMENT VERDICT

**NOT READY FOR HOSTILE
DEPLOYMENT**

2

CRITICAL VULNERABILITIES

7

HIGH SEVERITY FINDINGS

10

MEDIUM SEVERITY FINDINGS

4

LOW SEVERITY FINDINGS

1. Executive Summary

Systemic Posture & Core Axiom Assessment

Tetherless exhibits commendable engineering discipline in its cryptographic foundations: message bodies are protected with AES-256-GCM, DH keys are exchanged over RFC 3526 Group 14 with subgroup validation and left-padding normalisation, and the wire format relies on an explicit binary codec rather than Java deserialisation. However, the end-to-end security guarantee is undermined by several critical protocol and implementation vulnerabilities: inbound frames are never validated against the recipient's own client ID, signatures on control frames (`DELIVERY_ACK` , `READ_RECEIPT` , `TYPING` , `KEY_EXCHANGE_REJECT`) are completely bypassed when the signature is omitted, inbound GCM nonce direction bits are never verified, and TLS hostname verification is disabled on raw socket handshakes. Furthermore, unauthenticated relay registration permits trivial identity squatting and denial-of-service, and a null payload on acknowledgement frames crashes the client reader loop. In its current state, Tetherless is **not ready for production or hostile deployments** until these fundamental authentication, routing, and lifecycle defects are resolved.

Audit Distribution Matrix

Findings by Technical Domain

Section	Audit Domain	Critical	High	Medium	Low	Total
3.1	§3.1 Cryptography & Protocol Primitives, RFC 3526 DH key exchange, nonces, key storage, and wire signatures.	2	2	3	2	9
3.2	§3.2 Relay / Server Architecture Dumb relay queuing, connection registration, socket lifecycle, and network isolation.	0	2	1	1	4
3.3	§3.3 Client Correctness & State Desktop and mobile state machines, ack processing, message deduplication, and parsing.	0	2	1	0	3
3.4	§3.4 Input Validation & Injection Wire codecs, character sets, file permissions, and boundary deserialization.	0	0	1	0	1
3.5	§3.5 Build, Supply Chain & Secrets Dependencies, CVE exposure, release build gates, and SpotBugs static analysis filters.	0	1	2	0	3
3.6	§3.6 Tests & Verification Test vector completeness, adversarial coverage, stress tests, and lifecycle assertions.	0	0	1	1	2
3.7	§3.7 Architecture, Maintainability & Docs Documentation fidelity, specification drift, and code maintainability.	0	0	1	0	1
Total Audit Findings Across All Domains		2	7	10	4	23

Audit Scope & Verification Methodology: Complete architectural and source code security review of the Tetherless multi-module codebase (`core-shared` , `chat-server` , `chat-desktop`). Verification evaluated cryptographic correctness (AES-256-GCM nonces, RFC 3526 DH group validation, HKDF derivation), wire framing integrity, state-machine monotonicity, relay isolation against actively hostile operators, supply chain dependencies, and build gates. Automated build and regression verification passed cleanly across all modules (`:core-shared:check` , `:chat-server:check` , `:chat-desktop:check` , `:chat-desktop:integTest`).

§3.1 Cryptography & Protocol

Primitives, RFC 3526 DH key exchange, nonces, key storage, and wire signatures. — 9 findings identified in this domain.

Finding 3.1.1 Inbound Receiver ID Is Never Validated Across the Client Stack

CRITICAL

CONFIRMED

Location: core-shared/src/main/java/com/e2eechat/core/session/SessionManager.java:42-113

core-shared/src/main/java/com/e2eechat/core/session/SecureChat.java:189-220

chat-desktop/src/main/java/com/e2eechat/desktop/ChatClient.java:223-289

WHAT IS WRONG (DEFECT MECHANICS)

When a client receives a frame, neither `SessionManager`, `SecureChat`, nor `ChatClient` checks whether `msg.getReceiverId()` equals the local device's `clientId`.

CONCRETE FAILURE SCENARIO

- Bob sends a valid `KEY_EXCHANGE_INIT` or unencrypted control frame (`DELIVERY_ACK`, `READ_RECEIPT`) intended for Charlie (`receiverId = Charlie`, `senderId = Bob`).
- A malicious or misconfigured relay forwards this frame to Alice (`clientId = Alice`).
- Alice verifies Bob's signature against `msg.canonicalBytesForSigning()` (which passes, as Bob genuinely signed `receiverId = Charlie`).
- Alice's client does not verify that `msg.getReceiverId()` matches Alice, processes the frame under Alice's session with Bob, advances Alice's handshake or marks Alice's messages as delivered/read, and can trigger cross-session handshake collision or state corruption.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Message recipient binding is a fundamental invariant of any secure messaging protocol. Failing to enforce it permits cross-session reflection, misrouting attacks, and state confusion.

REMEDIATION GUIDANCE (HOW TO FIX)

In `SessionManager.onMessage`, add a strict assertion rejecting any frame where `msg.getReceiverId() != null && !localClientId.equals(msg.getReceiverId())`.

Finding 3.1.2

Signature Verification Completely Bypassed on Control & Ack Frames When Signature Is Absent

CRITICAL

CONFIRMED

Location: core-shared/src/main/java/com/e2eechat/core/protocol/SignatureVerifier.java:19-27

WHAT IS WRONG (DEFECT MECHANICS)

In `SignatureVerifier.verify()`:

```
boolean requiresSignature = (msg.getType() == MessageType.TEXT_MESSAGE ||
                             msg.getType() == MessageType.KEY_EXCHANGE_INIT ||
                             msg.getType() == MessageType.KEY_EXCHANGE_REPLY);

if (msg.getSignature() == null || msg.getSignature().length == 0) {
    return requiresSignature ? VerificationResult.MISSING_SIGNATURE : VerificationResult.VALID;
}
```

If `msg.getSignature()` is null or empty, `SignatureVerifier` returns `VerificationResult.VALID` for any message type other than `TEXT_MESSAGE`, `KEY_EXCHANGE_INIT`, and `KEY_EXCHANGE_REPLY`.

CONCRETE FAILURE SCENARIO

1. An active relay operator fabricates an unsigned frame with `type = DELIVERY_ACK`, `senderId = Bob`, `receiverId = Alice`, `payload = <messageId>`, and `signature = null`.
2. Alice receives the frame. `SessionManager.onMessage` invokes `SignatureVerifier.verify`, which returns `VALID`.
3. `ChatClient.onDelivered` executes `messageRepository.updateStatus(messageId, Status.DELIVERED)`.
4. Alice sees two checkmarks indicating Bob received the message, when Bob never received it. A hostile relay can similarly forge `READ_RECEIPT` (falsely marking conversations read), spoof `TYPING`, or send `KEY_EXCHANGE_REJECT` to terminate handshakes without holding Bob's private key.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Directly contradicts `docs/protocol.md §5` and `docs/security.md §5`, which claim all control frames are signed. Destroys peer authentication for message delivery and receipt status.

REMEDIATION GUIDANCE (HOW TO FIX)

Include all inter-peer control types (`DELIVERY_ACK`, `READ_RECEIPT`, `TYPING`, `KEY_EXCHANGE_REJECT`) in `requiresSignature`.

Finding 3.1.3

Nonce Direction Bit Is Ignored on Inbound Messages

HIGH

CONFIRMED

Location: core-shared/src/main/java/com/e2eechat/core/session/SessionManager.java:96-102

WHAT IS WRONG (DEFECT MECHANICS)

`SessionManager` parses `int direction = bb.getInt();` from the 12-byte IV, but never verifies `direction` against the expected peer direction bit.

CONCRETE FAILURE SCENARIO

1. Alice (`lowSide = true`, transmitting on `direction = 1`) sends message *M* to Bob.
2. If an adversary reflects Alice's message or if a corrupted/malicious peer sends frames with `direction = 1` rather than `direction = 0`, `SessionManager` ignores the mismatch.
3. The unverified counter is fed into `session.registerReceivedCounter(counter)` in `SessionManager.java:100`, consuming a counter slot in the receiver's replay window regardless of the direction bit used.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

The direction bit was explicitly designed to prevent GCM nonce collision between peers sharing a single key (`docs/protocol.md §9`). Failing to enforce the expected direction bit on the receive path permits direction-confusion and cross-direction counter exhaustion.

REMEDIATION GUIDANCE (HOW TO FIX)

In `SessionManager.onMessage`, compute `int expectedDirection = (localClientId.compareTo(msg.getSenderId()) > 0) ? 1 : 0;` and drop the frame if `direction != expectedDirection`.

Finding 3.1.4 TLS Hostname Verification Is Disabled on Client Sockets

HIGH

CONFIRMED

Location: `core-shared/src/main/java/com/e2echat/core/network/TlsSupport.java:235-241`

`core-shared/src/main/java/com/e2echat/core/network/ConnectionManager.java:228-231`

WHAT IS WRONG (DEFECT MECHANICS)

Sockets are created via `(SSLSocket) factory.createSocket(host, port)` with TLSv1.3 enabled, but `SSLParameters.setEndpointIdentificationAlgorithm("HTTPS")` is never called. Standard JSSE `SSLSocket` instances do **not** perform hostname verification against Subject Alternative Names (SAN) or CN by default.

CONCRETE FAILURE SCENARIO

1. An organization deploys a relay using a certificate from a trusted CA (e.g., Let's Encrypt, as instructed in `docs/deployment.md §2` and `docs/tls_provisioning.md`).
2. The client is configured with a truststore pinning the Let's Encrypt Root CA.
3. A network attacker on the client's local network (or upstream DNS) intercepts traffic to `relay.example.org` and presents any valid certificate issued by Let's Encrypt for `attacker-relay.com`.
4. Because endpoint identification is unset, the client verifies that the certificate chains to Let's Encrypt, ignores that the domain does not match `relay.example.org`, and establishes the TLS connection to the attacker's proxy.
5. Furthermore, this directly contradicts `docs/deployment.md:64-65`, which claims: *"A certificate without a matching SAN fails the handshake regardless of pinning."*

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

CA-pinned configurations are vulnerable to MITM interception by anyone possessing any certificate issued by the same CA.

REMEDIATION GUIDANCE (HOW TO FIX)

Configure `SSLParameters` `params = socket.getSSLParameters(); params.setEndpointIdentificationAlgorithm("HTTPS"); socket.setSSLParameters(params);` before starting the TLS handshake.

Finding 3.1.5

Replay Window Denial-of-Service via Counter Jump & Non-Positive Counter Acceptance

MEDIUM

CONFIRMED

Location: core-shared/src/main/java/com/e2eechat/core/session/Session.java:115-127

WHAT IS WRONG (DEFECT MECHANICS)

In `Session.registerReceivedCounter(long counter)`:

```
if (counter <= highestReceivedCounter - REPLAY_WINDOW) {
    return false;
}
if (!receivedCounters.add(counter)) {
    return false;
}
if (counter > highestReceivedCounter) {
    highestReceivedCounter = counter;
    forgetCountersBelowTheWindow();
}
return true;
```

The counter is not checked for `counter > 0` or bounded by the session send budget (`MAX_SENDS_PER_KEY = 100000`). When `highestReceivedCounter == 0`, negative counters and counter 0 satisfy `counter <= -1024 == false` and are accepted. More critically, if a peer transmits a validly signed message with counter `Long.MAX_VALUE`, `highestReceivedCounter` jumps to `Long.MAX_VALUE`.

CONCRETE FAILURE SCENARIO

An attacker (or compromised peer) injects a single validly signed message with `counter = Long.MAX_VALUE`. The recipient updates `highestReceivedCounter = Long.MAX_VALUE`. Thereafter, `floor = Long.MAX_VALUE - 1024`. All legitimate subsequent messages with normal counters (*1, 2, 3 |dots*) satisfy `counter <= floor` and are discarded permanently, completely bricking the session.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Allows permanent denial of service against a session using a single out-of-order counter jump.

REMEDATION GUIDANCE (HOW TO FIX)

Enforce `if (counter <= 0 || counter > MAX_SENDS_PER_KEY) return false;` and restrict maximum forward counter leap (`counter > highestReceivedCounter + REPLAY_WINDOW`).

Finding 3.1.6 Non-Atomic Peer Key Storage & Silent Loss of Pinning on Parsing Failure

MEDIUM

CONFIRMED

Location: `[`core-shared/src/main/java/com/e2eechat/core/keys/JceKeyStoreManager.java:47-54``110-116`](file:///c:/Users/aryan/Projects/tetherless/core-shared/src/main/java/com/e2eechat/core/keys/JceKeyStoreManager.java#L47-L54)`**WHAT IS WRONG (DEFECT MECHANICS)**

In `JceKeyStoreManager.storePeerKey`, the file `peers.properties` is overwritten directly using `new FileOutputStream(peersFile)`. If the JVM terminates mid-write, the file is corrupted or zeroed. On the next startup (lines 47–54), if reading `peers.properties` throws an exception, it logs `Could not read the stored peer keys... starting with none` and initializes an empty `peerProperties`.

CONCRETE FAILURE SCENARIO

A power failure or process crash occurs during `storePeerKey()`. On restart, `peers.properties` fails to parse. The client starts with an empty peer store. When known contacts subsequently reconnect and send `HELLO`, because `known.isPresent()` is false, the client silently re-trusts their presented keys on first use (TOFU) via `SecureChat.onHello` instead of raising `PEER_KEY_CHANGED`. If an active MITM occurs during this window, the impostor's key is pinned without any warning.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Silent reset of trust stores destroys the core TOFU guarantee.

REMEDIATION GUIDANCE (HOW TO FIX)

Write to a temporary file (`peers.properties.tmp`) and atomically move it (`Files.move(..., ATOMIC_MOVE)`). If the file exists but fails to load, fail closed (abort startup or alert user) rather than silently discarding all pinned keys.

Finding 3.1.7 Sensitive In-Memory DH Key Material Not Zeroed in Heap

MEDIUM

CONFIRMED

Location: `core-shared/src/main/java/com/e2eechat/core/crypto/DHUtils.java:78-91`**WHAT IS WRONG (DEFECT MECHANICS)**

`byte[] rawSecret = keyAgreement.generateSecret();` and `byte[] paddedSecret` in `DHUtils.generateSharedSecret` are discarded without calling `Arrays.fill(..., (byte) 0)`.

CONCRETE FAILURE SCENARIO

The un-zeroed 2048-bit Diffie-Hellman shared secret remains in JVM heap memory until collected and overwritten by garbage collection. In a multi-tenant process, crash dump, or heap inspection scenario, raw shared secrets can be recovered.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Incomplete memory hygiene for ephemeral master secrets.

REMEDIATION GUIDANCE (HOW TO FIX)

Enclose derivation in a `try ... finally` block and execute `Arrays.fill(rawSecret, (byte) 0)` and `Arrays.fill(paddedSecret, (byte) 0)`.

Finding 3.1.8 Non-Repudiation vs. Deniability Trade-off in Protocol Design

LOW

CONFIRMED

Location: `core-shared/src/main/java/com/e2eechat/core/models/Message.java:34-58`

`core-shared/src/main/java/com/e2eechat/core/protocol/MessageSigner.java:11-30`

WHAT IS WRONG (DEFECT MECHANICS)

Every text message is signed with the sender's long-term RSA identity private key (`SHA256withRSA`) over canonical bytes containing the timestamp, message ID, sender ID, receiver ID, and ciphertext.

CONCRETE FAILURE SCENARIO

A recipient (or anyone obtaining the recipient's decrypted database and the signed wire frames) can mathematically prove to a third party (e.g., in a legal or public context) that the specific sender identity signed that exact message at that timestamp. This provides strict cryptographic **non-repudiation**, which is the antithesis of **off-the-record deniability** (as provided by Signal's X3DH / Double Ratchet using MAC subkeys).

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Architectural mismatch for users expecting plausible deniability in an end-to-end encrypted chat application.

REMEDIATION GUIDANCE (HOW TO FIX)

Transition to a deniable authenticated key exchange (e.g., Signal-style X3DH with ephemeral Diffie-Hellman authenticator keys rather than long-term RSA digital signatures on individual chat payloads).

Finding 3.1.9 Dead Code: Orphaned and Incomplete `SessionKeyCache` Class

LOW

CONFIRMED

Location: `core-shared/src/main/java/com/e2eechat/core/keys/SessionKeyCache.java:1-26`

WHAT IS WRONG (DEFECT MECHANICS)

`SessionKeyCache` is listed in documentation and project audit prompt anchors, but is never referenced, instantiated, or used by any class across `core-shared`, `chat-server`, or `chat-desktop`.

CONCRETE FAILURE SCENARIO

Maintenance overhead and false assumption by auditors that key caching logic is managed by this class, whereas `SessionManager` directly maintains sessions.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Incomplete architecture and dead code cause confusion for security reviewers and increase maintenance debt.

REMEDIATION GUIDANCE (HOW TO FIX)

Remove the class or integrate it into `SessionManager`.

§3.2 Relay / Server Architecture

Dumb relay queuing, connection registration, socket lifecycle, and network isolation. — 4 findings identified in this domain.

Finding 3.2.1 Unauthenticated Client Registration Permitting Trivial DoS & Peer ID Squatting

HIGH

CONFIRMED

Location: `chat-server/src/main/java/com/e2eechat/server/ClientSession.java:202-221`

`core-shared/src/main/java/com/e2eechat/core/network/ConnectionManager.java:246-253`

WHAT IS WRONG (DEFECT MECHANICS)

In `ClientSession.run`, when the relay receives the opening `HELLO`, it extracts `clientId = message.getSenderId()` and calls `registry.register(clientId, this)`. The registration `HELLO` is unsigned (`ConnectionManager.java:252` uses `buildUnsigned()`), carries no public key, and includes no proof of possession of the private key matching the `PeerId`.

CONCRETE FAILURE SCENARIO

1. An attacker determines Alice's public 32-character hexadecimal `PeerId`.
2. The attacker connects to the relay over TLS and sends a 28-byte raw frame containing `HELLO` with `senderId = Alice` and `payload = null`.
3. The relay registers the attacker's socket under `Alice`.
4. When the legitimate Alice launches her client and attempts to connect, the relay finds `Alice` in `ClientRegistry`, rejects Alice's connection with `ID_TAKEN`, and disconnects her socket.
5. The attacker can script this for all known peer IDs, permanently locking all legitimate users out of the relay with minimal resources.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Severe denial-of-service and identity squatting on the relay.

REMEDIATION GUIDANCE (HOW TO FIX)

Require the registration `HELLO` to carry a challenge response, nonce signature, or self-signed payload proving possession of the private key whose public key hashes to `PeerId`.

Finding 3.2.2 Relay Routes Frames Without Verifying Sender Identity

HIGH

CONFIRMED

Location: `[`chat-server/src/main/java/com/e2eechat/server/ClientSession.java:238-240`

`258`

`272-296`](file:///c:/Users/aryan/Projects/tetherless/chat-server/src/main/java/com/e2eechat/server/ClientSession.java#L238-L240)`

WHAT IS WRONG (DEFECT MECHANICS)

In `ClientSession.java:258`, once a session is established, `routeFrame(message.getReceiverId(), frame)` forwards the raw encoded bytes without checking if `message.getSenderId().equals(this.clientId)`.

CONCRETE FAILURE SCENARIO

A connected client authenticated with `clientId = Mallory` sends a frame with `senderId = Alice` and `receiverId = Bob`. Combined with Finding 3.1.2 (where `DELIVERY_ACK`, `READ_RECEIPT`, and `TYPING` do not require signatures), Mallory can directly impersonate Alice's acknowledgements and typing events through the relay to Bob.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

The relay fails to enforce sender authenticity at the routing boundary.

REMEDIATION GUIDANCE (HOW TO FIX)

In `ClientSession.run()`, before calling `routeFrame`, assert `if (!this.clientId.equals(message.getSenderId())) { disconnect(); return; }`.

Finding 3.2.3 Prometheus Metrics Server Binds Wildcard (0.0.0.0) by Default

MEDIUM

CONFIRMED

Location: `chat-server/src/main/java/com/e2eechat/server/MetricsServer.java:25``[` chat-server/src/main/java/com/e2eechat/server/ChatServer.java:52``59` ](file:///c:/Users/aryan/Projects/tetherless/chat-server/src/main/java/com/e2eechat/server/ChatServer.java#L52)`**WHAT IS WRONG (DEFECT MECHANICS)**

In `MetricsServer.start()`, `HttpServer.create(new InetSocketAddress(port), 0)` binds to the wildcard interface (0.0.0.0), exposing `/metrics` over unauthenticated HTTP on all network interfaces.

CONCRETE FAILURE SCENARIO

An operator runs `tetherless-relay` on a public VPS without a local firewall. Anyone on the internet connecting to `http://<relay-ip>:<PORT+1>/metrics` can read real-time operational telemetry: active client count, messages routed, buffer overflows, and rejection counters. This contradicts `docs/deployment.md:142-144`, which warns that metrics must not be accessible externally.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Unauthenticated information disclosure of operational and traffic patterns.

REMEDIATION GUIDANCE (HOW TO FIX)

Bind explicitly to loopback: `new InetSocketAddress(InetAddress.getLoopbackAddress(), port)`, or expose a `METRICS_HOST` configuration property defaulting to `127.0.0.1`.

Finding 3.2.4 Hardcoded Default Keystore Password in ServerConfig

LOW

CONFIRMED

Location: `[` chat-server/src/main/java/com/e2eechat/server/ServerConfig.java:25``28``63-65` ](file:///c:/Users/aryan/Projects/tetherless/chat-server/src/main/java/com/e2eechat/server/ServerConfig.java#L25)`**WHAT IS WRONG (DEFECT MECHANICS)**

`ServerConfig.java` defines `static final String DEFAULT_KEYSTORE_PASSWORD = "changeit";` as a literal, and does not synchronize with `TlsSupport.trustStorePassword()`.

CONCRETE FAILURE SCENARIO

If a developer sets `TETHERLESS_TRUSTSTORE_PASSWORD` in the environment as instructed by `README.md:75`, `ServerConfig` does not read that variable. It falls back to literal `"changeit"`, failing to start if the keystore was generated with that password.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Inconsistency between client/server configuration and reliance on hardcoded password literals.

REMEDIATION GUIDANCE (HOW TO FIX)

Refactor `ServerConfig` to delegate fallback password retrieval to `TlsSupport.trustStorePassword()`.

§3.3 Client Correctness & State

Desktop and mobile state machines, ack processing, message deduplication, and parsing. — 3 findings identified in this domain.

Finding 3.3.1

Unguarded Message Status Transition Allows Status Regression from **READ** to

DELIVERED

HIGH

CONFIRMED

Location: chat-desktop/src/main/java/com/e2eechat/desktop/MessageRepository.java:87-100

chat-desktop/src/main/java/com/e2eechat/desktop/ChatClient.java:279-287

WHAT IS WRONG (DEFECT MECHANICS)

In `MessageRepository.updateStatus` :

```
PreparedStatement pstmt = conn.prepareStatement(
    "UPDATE messages SET status = ? WHERE message_id = ?");
pstmt.setString(1, status.name());
pstmt.setString(2, messageId);
pstmt.executeUpdate();
```

There is no guard against backward state transitions in the delivery lifecycle.

CONCRETE FAILURE SCENARIO

1. Alice sends message M_1 to Bob.
2. Bob reads the message; Bob's client sends a `READ_RECEIPT`.
3. Alice's client receives the receipt, executing `markOutgoingRead`, which transitions M_1 from `SENT` to `READ`.
4. A delayed or replayed `DELIVERY_ACK` for M_1 arrives later (due to network latency, reconnection, or relay retry).
5. `ChatClient.onDelivered` executes `messageRepository.updateStatus(mId, Status.DELIVERED)`.
6. M_1 's status in the local SQLite database is overwritten from `READ` back to `DELIVERED`. The UI downgrades the double checkmark back to a single/delivered checkmark.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Violates state machine monotonicity and corrupts message status presentation.

REMEDIATION GUIDANCE (HOW TO FIX)

Guard the SQL update:

```
UPDATE messages SET status = ? WHERE message_id = ? AND (status != 'READ' OR ? = 'READ') or enforce monotonic ordinal progression ( status_rank(new) > status_rank(current) ).
```

Finding 3.3.2

Null Payload on Control/Ack Frames Crashes Reader Loop & Drops Network Connection

HIGH

CONFIRMED

Location: chat-desktop/src/main/java/com/e2echat/desktop/ChatClient.java:284-286

core-shared/src/main/java/com/e2echat/core/network/ConnectionManager.java:303-311

[chat-desktop/src/main/java/com/e2echat/desktop/ChatWindow.java:286

313`](file:///c:/Users/aryan/Projects/tetherless/chat-desktop/src/main/java/com/e2echat/desktop/ChatWindow.java#L286)

WHAT IS WRONG (DEFECT MECHANICS)

`MessageCodec.decode` permits null payloads (`payload.length == -1`). However, `ChatClient.onDelivered` directly invokes `new String(msg.getPayload(), StandardCharsets.UTF_8)` when handling `DELIVERY_ACK` . Similarly, `ChatWindow.handleMessage` calls `new String(msg.getPayload(), StandardCharsets.UTF_8)` on `ERROR` and `DELIVERY_ACK` .

CONCRETE FAILURE SCENARIO

1. A peer (or the relay) transmits a wire frame of type `DELIVERY_ACK` with `payload = null` (`-1` in wire encoding).
2. `ChatClient.onMessageReceived` runs on the `Client-Reader` thread and calls `onDelivered()` .
3. `new String(null, ...)` throws a `NullPointerException` .
4. The unhandled exception bubbles up to `ConnectionManager.readerLoop` , which catches `Exception` , logs `Reader error: null` , and terminates by calling `own.close()` .
5. The entire network connection is severed. A malicious peer can continuously disconnect a user by sending a single 20-byte null-payload frame.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Remotely triggerable denial of service and client connection drop.

REMEDIATION GUIDANCE (HOW TO FIX)

Add null checks (`if (msg.getPayload() == null) return;`) in `ChatClient.onDelivered` and `ChatWindow.handleMessage` .

Finding 3.3.3

Delimiter Injection in Quoted Reply Encoding

MEDIUM

CONFIRMED

Location: chat-desktop/src/main/java/com/e2echat/desktop/ChatClient.java:549-579

WHAT IS WRONG (DEFECT MECHANICS)

In `Body.encode` :

```
return REPLY_MARKER + replyId + FIELD_SEP
    + (replySender == null ? "" : replySender) + FIELD_SEP
    + (replyPreview == null ? "" : replyPreview.replace('\n', ' ')) + BODY_SEP
    + text;
```

`FIELD_SEP` (`\u001F`) and `BODY_SEP` (`\u001E`) are used as framing delimiters. `replyPreview.replace('\n', ' ')` sanitizes newlines, but does **not** escape or strip `\u001F` or `\u001E` from `replyPreview` , `replySender` , or `replyId` .

CONCRETE FAILURE SCENARIO

A user sends a message containing `\u001E` or `\u001F` . When a contact quotes that message in a reply, `Body.encode` embeds the raw delimiter. On the receiving end, `Body.parse` splits on `FIELD_SEP` or matches `BODY_SEP` prematurely, corrupting the message text or displacing the reply preview into the message body.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Message formatting corruption and potential display text injection via quoted reply metadata.

REMEDIATION GUIDANCE (HOW TO FIX)

Strip or escape `\u001E` and `\u001F` from all metadata fields in `Body.encode` .

\$3.4 Input Validation & Injection

Wire codecs, character sets, file permissions, and boundary deserialization. — 1 findings identified in this domain.

Finding 3.4.1 Missing POSIX File Permissions on Private Key & Peer Stores

MEDIUM

CONFIRMED

Location: [`core-shared/src/main/java/com/e2echat/core/keys/JceKeyStoreManager.java:103`

113`](file:///c:/Users/aryan/Projects/tetherless/core-shared/src/main/java/com/e2echat/core/keys/JceKeyStoreManager.java#L103)

[`chat-desktop/src/main/java/com/e2echat/desktop/PeerDirectory.java:150`

160`](file:///c:/Users/aryan/Projects/tetherless/chat-desktop/src/main/java/com/e2echat/desktop/PeerDirectory.java#L150)

WHAT IS WRONG (DEFECT MECHANICS)

When creating `identity.p12`, `peers.properties`, and `peer-verification.properties`, standard `FileOutputStream` is used without setting POSIX permissions (e.g. `0600`).

CONCRETE FAILURE SCENARIO

On multi-user Linux or macOS systems, files created with default umask (e.g., `0644`) are world-readable. Any local user account on the machine can read `identity.p12` to mount an offline dictionary attack against the passphrase, or read `peers.properties` to inspect the user's entire contact list.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Local privilege boundary leakage on multi-user systems.

REMEDIATION GUIDANCE (HOW TO FIX)

Use `File.setReadable(false, false); File.setReadable(true, true);` or POSIX file attribute permissions (`PosixFilePermissions.asFileAttribute(...)`) when creating sensitive files and parent configuration directories.

\$3.5 Build, Supply Chain & Secrets

Dependencies, CVE exposure, release build gates, and SpotBugs static analysis filters. — 3 findings identified in this domain.

Finding 3.5.1 Known Remote Code Execution Vulnerability in `sqlite-jdbc` (CVE-2023-32697)

HIGH

CONFIRMED

Location: [`gradle/libs.versions.toml:10`

24`](file:///c:/Users/aryan/Projects/tetherless/gradle/libs.versions.toml#L10)

WHAT IS WRONG (DEFECT MECHANICS)

The build pins `sqlite-jdbc = "3.41.2.1"`. Version 3.41.2.1 is vulnerable to **CVE-2023-32697** (High severity), an arbitrary code execution vulnerability affecting `sqlite-jdbc` versions through 3.41.2.1. The flaw was resolved in version **3.41.2.2**.

CONCRETE FAILURE SCENARIO

Any path where connection properties or database URLs can be influenced by configuration or untrusted input exposes the JVM to arbitrary code execution via crafted JDBC URLs.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Packaging a dependency known to be vulnerable to Remote Code Execution violates supply chain security standards.

REMEDIATION GUIDANCE (HOW TO FIX)

Update `sqlite-jdbc` in `gradle/libs.versions.toml` to `3.45.1.0` or later (at minimum `3.41.2.2`).

Finding 3.5.2

Known Vulnerabilities in Declared

logback-classic

Dependency

MEDIUM

CONFIRMED

Location: `[` gradle/libs.versions.toml:4 18`](file:///c:/Users/aryan/Projects/tetherless/gradle/libs.versions.toml#L4)`

WHAT IS WRONG (DEFECT MECHANICS)

The build pins `logback = "1.5.0"`. Logback 1.5.0 contains known security vulnerabilities (including CVE-2025-11226 and related configuration/deserialization issues) patched in subsequent 1.5.x and 1.6.x releases.

CONCRETE FAILURE SCENARIO

Exploitation of logback configuration parsing or appender vulnerabilities in environments where logging configuration or environment variables are controlled by external entities.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Supply chain hygiene for core logging dependencies.

REMEDiation GUIDANCE (HOW TO FIX)

Update `logback` to `1.5.19` or higher.

Finding 3.5.3

Blanket Global Suppressions in SpotBugs Filter Contradicting Stated Security Policy

MEDIUM

CONFIRMED

Location: `[` config/spotbugs/exclude.xml:15-17 23-25`](file:///c:/Users/aryan/Projects/tetherless/config/spotbugs/exclude.xml#L15-L17)` `docs/security.md:337-340`

WHAT IS WRONG (DEFECT MECHANICS)

`docs/security.md §8` states:

> "Three suppressions exist, each scoped to a single pattern or method and each carrying its reason in `config/spotbugs/exclude.xml`; there is no blanket exclusion, because on a security detector a pattern-wide suppression silently covers code nobody has looked at yet."

In reality, `config/spotbugs/exclude.xml` contains blanket exclusions:

```
<Match>
  <Bug pattern="OBJECT_DESERIALIZATION" />
</Match>
<Match>
  <Bug pattern="DM_DEFAULT_ENCODING" />
</Match>
```

Neither match is scoped to a class or method.

CONCRETE FAILURE SCENARIO

A developer introduces a call to `ObjectInputStream.readObject()` on an untrusted stream anywhere in the codebase. SpotBugs fails to flag it because `OBJECT_DESERIALIZATION` is globally suppressed.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Discrepancy between published security guarantees and static analysis enforcement.

REMEDiation GUIDANCE (HOW TO FIX)

Remove the blanket `<Bug pattern="OBJECT_DESERIALIZATION" />` suppression. If a specific class genuinely needs an exclusion, scope it explicitly by `<Class name="..." />` and `<Method name="..." />`.

§3.6 Tests & Verification

Test vector completeness, adversarial coverage, stress tests, and lifecycle assertions. — 2 findings identified in this domain.

Finding 3.6.1 Test Suite Fails to Exercise Tampering on Control & Ack Frames

MEDIUM

CONFIRMED

Location: `core-shared/src/test/java/com/e2eechat/core/session/AdversarialRelayTest.java:174-186`

WHAT IS WRONG (DEFECT MECHANICS)

In `AdversarialRelayTest.strippingTheSignatureDropsTheMessage`:

```
relay.setTamper(message -> message.getType() == MessageType.TEXT_MESSAGE
    ? MaliciousRelay.stripSignature(message)
    : message);
```

The test only verifies stripping signatures on `TEXT_MESSAGE`. It completely omits testing whether unsigned `DELIVERY_ACK`, `READ_RECEIPT`, or `TYPING` frames are dropped.

CONCRETE FAILURE SCENARIO

Because tests only tested `TEXT_MESSAGE`, the failure of `SignatureVerifier.java:26` to enforce signatures on control frames went unnoticed and passed all CI checks.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

False sense of test coverage on adversarial relay capabilities.

REMEDIATION GUIDANCE (HOW TO FIX)

Extend `AdversarialRelayTest` to assert that stripping signatures on `DELIVERY_ACK`, `READ_RECEIPT`, `TYPING`, and `KEY_EXCHANGE_REJECT` causes them to be dropped.

Finding 3.6.2 `IvReuseTest` Iteration Count Is Orders of Magnitude Below Key Budget

LOW

CONFIRMED

Location: `core-shared/src/test/java/com/e2eechat/core/session/IvReuseTest.java:43-58`

`core-shared/src/main/java/com/e2eechat/core/session/Session.java:88`

WHAT IS WRONG (DEFECT MECHANICS)

`IvReuseTest.java` runs only 20 iterations across directions and 200 iterations for a single sender. While mathematically the counter increments deterministically, it does not test behavior near `MAX_SENDS_PER_KEY` (100,000) or verify IV generation across multiple session renewal cycles in a single test.

CONCRETE FAILURE SCENARIO

A regression in counter reset or direction assignment across multiple consecutive key renewals could go undetected.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Inadequate stress testing of the primary AES-GCM invariant.

REMEDIATION GUIDANCE (HOW TO FIX)

Add a parameterized test simulating renewal transitions across several consecutive keys, asserting distinct nonces across the entire lifecycle.

§3.7 Architecture, Maintainability & Docs

Documentation fidelity, specification drift, and code maintainability. — 1 findings identified in this domain.

Finding 3.7.1

Discrepancies Between Documentation Guarantees and Code Implementation

MEDIUM

CONFIRMED

Location: docs/deployment.md:64-65 [docs/security.md:184

337`](file:///c:/Users/aryan/Projects/tetherless/docs/security.md#L184) docs/protocol.md:91-106

WHAT IS WRONG (DEFECT MECHANICS)

1. docs/deployment.md:64 claims: "A certificate without a matching SAN fails the handshake regardless of pinning." In reality, hostname verification is not enabled on the client socket (Finding 3.1.4).

2. docs/security.md:184 and docs/protocol.md §5 state that typing notifications and read receipts are signed. In reality, unsigned versions are accepted as valid by SignatureVerifier (Finding 3.1.2).

3. docs/security.md:337 claims every SpotBugs suppression is scoped to a single pattern or method without blanket exclusions. In reality, blanket exclusions exist (Finding 3.5.3).

CONCRETE FAILURE SCENARIO

Security auditors and operators relying on documentation make incorrect security assumptions regarding MITM protection, signature integrity, and static analysis rigor.

WHY IT MATTERS (CRYPTOGRAPHIC & SYSTEMIC IMPACT)

Over-claiming in security documentation misleads users and auditors.

REMEDIATION GUIDANCE (HOW TO FIX)

Align documentation with code or implement the missing checks.

3. Confirmed Strengths

7 Independently Verified Security Controls

The following architectural mechanisms and cryptographic controls were verified in the source code as effective implementations conforming to secure design principles:

VERIFIED #1 No Java Serialization on the Wire:

`core-shared/src/main/java/com/e2eechat/core/protocol/MessageCodec.java` implements a hand-rolled binary codec with strict length checks before any allocation (`readByteArray` enforces bounds before array creation). No wire models implement `java.io.Serializable`.

VERIFIED #2 MODP Group 14 Subgroup Validation:

`core-shared/src/main/java/com/e2eechat/core/crypto/DHUtils.java:62-72` strictly enforces bounds $1 < y < p-1$ and tests $y^q \bmod p == 1$, preventing small-subgroup attacks.

VERIFIED #3 Leading Zero Normalisation in Diffie-Hellman Secret Derivation:

`core-shared/src/main/java/com/e2eechat/core/crypto/DHUtils.java:80-89` correctly left-pads the raw DH secret to 256 bytes, preventing intermittent 1-in-256 handshake derivation failures between JDK and Conscript providers.

VERIFIED #4 Frozen Protocol Vectors & Cross-Platform Conformance:

`core-shared/src/main/java/com/e2eechat/core/protocol/ProtocolVectors.java` and `ProtocolConformance.java` verify 17 frozen protocol test vectors and RFC 5869 HKDF vectors, preventing silent protocol drift.

VERIFIED #5 Release Build Certificate Gating:

`core-shared/src/main/java/com/e2eechat/core/network/TlsSupport.java:123-129` and `chat-server/src/main/java/com/e2eechat/server/ServerConfig.java:149-156` actively inspect `BuildInfo.isRelease()`. Packaged release builds hard-fail if unconfigured rather than falling back to the development keystore.

VERIFIED #6 In-Process Identity Key Generation:

`core-shared/src/main/java/com/e2eechat/core/keys/JceKeyStoreManager.java:88-107` generates RSA identity keys and self-signed certificates entirely in-process using `SelfSignedCertificate.java`, eliminating command-line argument exposure via `keytool`.

VERIFIED #7 Display Name Sanitization:

`core-shared/src/main/java/com/e2eechat/core/protocol/HelloPayload.java:130-145` strips ASCII control characters, Unicode bidirectional overrides (`U+202A..U+202E`), and isolates (`U+2066..U+2069`) before rendering.

4. Documented Limitations Confirmed Present

6 Verified Boundary Non-Goals

The following documented limitations from `docs/security.md §5` were independently verified as present in the implementation:

LIMITATION #1 No Forward Secrecy / No Ratchet:

Confirmed in `SecureChat.java:132-155` and `Session.java:88`. A single DH exchange generates a key that persists for up to 100,000 messages. There is no Double Ratchet or per-message key rotation.

LIMITATION #2 Absence of Metadata Privacy from the Relay:

Confirmed in `chat-server/src/main/java/com/e2eechat/server/ClientSession.java:272-296`. The relay observes full communication metadata (sender, receiver, frame length, and timestamps).

LIMITATION #3 Startup GitHub Update Check:

Confirmed in `chat-desktop/src/main/java/com/e2eechat/desktop/UpdateChecker.java:108-144`. Performs an outbound HTTPS request to GitHub's release API on startup unless disabled via `updates=false`.

LIMITATION #4 Partial At-Rest Encryption on Desktop:

Confirmed in `chat-desktop/src/main/java/com/e2eechat/desktop/MessageRepository.java:55-84`. Only message text and reply preview columns are AES-GCM encrypted; `sender`, `receiver`, `timestamp`, and message counts remain plaintext in SQLite.

LIMITATION #5 No Session Expiry / Time-to-Live:

Confirmed in `Session.java:11` . `Session.State.EXPIRED` is never assigned.

LIMITATION #6 Single Centralized Relay Architecture:

Confirmed across `chat-server` . No peer-to-peer or federated relay capabilities are implemented.

5. Top 5 Priorities to Fix First

To achieve baseline cryptographic integrity and prevent active relay compromise, the following 5 remediation items must be implemented prior to release:

Rank	Finding ID	Severity	File Reference	Action Required
1	Finding 3.1.2	CRITICAL	SignatureVerifier.java:20-27	Enforce signatures on all control frame types (DELIVERY_ACK , READ_RECEIPT , TYPING , KEY_EXCHANGE_REJECT).
2	Finding 3.1.1	CRITICAL	SessionManager.java:42-66	Assert localClientId.equals(msg.getReceiverId()) on all incoming frames to prevent cross-session misrouting.
3	Finding 3.2.1	HIGH	ClientSession.java:202-221	Authenticate registration HELLO with a cryptographic signature/proof-of-possession to prevent trivial identity squatting.
4	Finding 3.1.4	HIGH	TlsSupport.java:235-241	Enable TLS endpoint identification (params.setEndpointIdentificationAlgorithm("HTTPS")) on all raw SSLSocket instances.
5	Finding 3.3.1 & 3.3.2	HIGH	MessageRepository.java:92-96 , ChatClient.java:284-286	Guard status updates against regression from READ to DELIVERED , and null-check payloads to prevent connection drops.